

MLP + Batch Normalization

The Math Explained

A complete derivation of every formula in *build_makemore_batchnorm.ipynb* — embeddings, Kaiming initialisation, BatchNorm forward and backward pass, tanh saturation, running statistics, parameter updates, and the deeper 6-layer network.

Andrej Karpathy · makemore series · Part 3

Based on the original Colab notebook [build_makemore_batchnorm.ipynb](#)

§	Section	Topic
1	Architecture Overview	The full pipeline — $C \rightarrow W1 \rightarrow BN \rightarrow \tanh \rightarrow W2 \rightarrow \text{loss}$
2	Embeddings (C)	Lookup table, one-hot equivalence, shapes
3	Linear Layer W1	Kaiming init, variance analysis, $(5/3)/\sqrt{n}$
4	Batch Normalization	Three steps: compute, normalise, scale+shift
5	BN Parameters γ, β	Why 1 and 0, learnable affine transform
6	Running Statistics	EMA formula, training vs inference mode
7	Tanh & Saturation	Derivative, gradient health table, 0.97 threshold
8	Output Layer & Loss	W2 init, cross-entropy formula
9	Full Forward Pass	One combined formula, 7-step pipeline
10	Backpropagation	Gradients through every layer including BN
11	Parameter Update	SGD step, learning-rate decay
12	Numerical Examples	Step-by-step numbers for BN forward + backward
13	Custom Classes	Linear, BatchNorm1d, Tanh — PyTorch API match

14 Deeper 6-Layer Network	Stacking blocks, bias=False, output layer init
15 Inference & Sampling	eval mode, running stats, name generation
16 Linear vs Linear+BN	std=31 vs std=1 experiment explained

1 Architecture Overview

This notebook extends the basic MLP character language model with **Batch Normalization**. The forward pipeline has five stages, then a loss:

Stage	Operation	Shape (batch B=32)	Notes
Embedding	$C[Xb]$	(32, 3, 10)	Lookup 3 chars $\times$ 10-dim vectors
Concat	<code>emb.view(B, -1)</code>	(32, 30)	Flatten context
Linear W1	<code>embcat @ W1</code>	(32, 200)	No bias when BN follows
BatchNorm	$\gamma \cdot x_{\text{BN}} + \beta$	(32, 200)	Normalise + learnable scale/shift
Tanh	<code>tanh(hBN)</code>	(32, 200)	Values in (-1, +1)
Linear W2	<code>h @ W2 + b2</code>	(32, 27)	27 = vocab size
Loss	<code>cross_entropy(logits, y)</code>	scalar	Avg negative log-prob

2 Embeddings (C)

What: Every character index i maps to a dense 10-dimensional learned vector. C is a (27×10) lookup table.

$$e_i = C[i] \in \mathbb{R}^{10}$$

Why not one-hot? One-hot vectors are 27-dimensional and sparse — they carry no notion of similarity. Dense embeddings let the network discover that vowels are similar, consonants cluster, etc., enabling generalisation to unseen contexts.

Mathematically $C[Xb]$ is identical to `one_hot(Xb) @ C` — indexing is just a fast matmul with a one-hot matrix. PyTorch verifies: `torch.allclose(F.one_hot(x) @ C, C[x]) == True`

Shapes: C : (27, 10) | $C[Xb]$: (32, 3, 10) | `embcat = emb.view(B, -1)`: (32, 30)

3 Linear Layer W1 — Kaiming Initialisation

The first linear layer maps the 30-dim flattened embedding to 200 hidden neurons:

$$h_{\text{pre}} = \text{embcat} \cdot W1 \quad \text{where } W1 \in \mathbb{R}^{(30 \times 200)}$$

Initialisation code:

```
W1 = torch.randn((30, 200), generator=g) * (5/3) / (30**0.5)
```

Why this specific scale?

- 1 If $w \sim N(0,1)$ and input x has unit variance, the output $h = xW$ has variance $\text{Var}(h) = n_{\text{in}} \cdot \text{Var}(w)$. To keep $\text{Var}(h) = 1$ we need $\text{Var}(w) = 1/n_{\text{in}}$, i.e. scale by $1/\sqrt{n_{\text{in}}}$.
- 2 For tanh, a plain $1/\sqrt{n_{\text{in}}}$ (Xavier) produces slight saturation. The factor $5/3$ is the empirically derived tanh gain — it corrects for the slight compression tanh applies and keeps most inputs in the linear regime.

$W1 \sim N(0, ((5/3)/\sqrt{n_{\text{in}}})^2)$ where $n_{\text{in}} = n_{\text{embd}} \times \text{block_size} = 30$

Initialisation	Std of h_{pre}	Tanh saturation	Gradient health
Raw randn (no scale)	~5.5	~85%	Very weak
Xavier $1/\sqrt{n}$	~1.0	~28%	Moderate
Kaiming-tanh $(5/3)/\sqrt{n}$	~1.67	~5%	Strong

4 ~~Batch Normalization — Three-Step Formula~~

Introduced by Ioffe & Szegedy (2015). Applied after the linear layer, before tanh.

Step 1 — Compute batch statistics (per feature, across all m samples in the mini-batch):

$$\mu = (1/m) \sum_i h_{\text{pre},i}$$

$$\sigma^2 = (1/m) \sum_i (h_{\text{pre},i} - \mu)^2$$

Code: `bnmean = hpreact.mean(0, keepdim=True)` `bnstd = hpreact.std(0, keepdim=True)`

Step 2 — Normalise (z-score each value):

$$\hat{h}_i = (h_{\text{pre},i} - \mu) / \sqrt{(\sigma^2 + \epsilon)} \quad \text{where } \epsilon = 1e-5$$

ϵ (epsilon) prevents division by zero when $\sigma^2 \approx 0$. After this step: mean ≈ 0 , variance ≈ 1 .

Step 3 — Scale and shift (learnable affine transform):

$$h_{\text{BN},i} = \gamma \cdot \hat{h}_i + \beta$$

γ (bngain) and β (bnbias) are learnable parameters updated by backprop. This gives the network the freedom to undo normalisation if beneficial — but without losing the gradient stability BN provides.

Why both steps? Step 2 alone forces every layer to mean=0, var=1 always — too restrictive. The learnable γ and β in step 3 restore expressiveness while still keeping gradients stable through normalisation.

5 BN Parameters. γ (bngain) and β (bnbias)

Parameter	Symbol	Init	Role	Effect if changed
bngain	γ	1.0	Scale normalised activations	$\gamma > 1$ amplifies; $\gamma < 1$ shrinks
bnbias	β	0.0	Shift normalised activations	Moves distribution up/down
bnmean_running	μ_{run}	0.0	Global mean for inference	Updated by EMA
bnstd_running	σ_{run}	1.0	Global std for inference	Updated by EMA

Why $\gamma=1$ and $\beta=0$? This is the identity initialisation. At the start of training, BN outputs exactly the normalised values. Training then discovers the optimal γ and β through gradient descent.

Numerical example — normalised values $\hat{h} = [-1.0, 0.0, +1.0]$:

γ	β	$h_{\text{BN}} = \gamma \hat{h} + \beta$	$\tanh(h_{\text{BN}})$	$\tanh$ derivative	Comment
1	0	$[-1.0, 0.0, +1.0]$	$[-0.76, 0.00, +0.76]$	$[0.42, 1.00, 0.42]$	Healthy
2	0	$[-2.0, 0.0, +2.0]$	$[-0.96, 0.00, +0.96]$	$[0.07, 1.00, 0.07]$	More saturation
1	1	$[0.0, +1.0, +2.0]$	$[0.00, +0.76, +0.96]$	$[1.00, 0.42, 0.07]$	Biased positive

6 Running Statistics — Training vs. Inference

The problem: During training, BN uses the current batch's mean and variance. At inference, you may predict one sample at a time — a batch of 1 has no meaningful variance. Solution: track running averages during training and use them at inference.

Exponential Moving Average (EMA) update inside training loop:

```

 $\mu_{\text{run}} \leftarrow (1 - \alpha) \cdot \mu_{\text{run}} + \alpha \cdot \mu_{\text{batch}}$ 
 $\sigma_{\text{run}} \leftarrow (1 - \alpha) \cdot \sigma_{\text{run}} + \alpha \cdot \sigma_{\text{batch}}$ 
( $\alpha = 0.001$  in the inline code;  $\alpha = 0.1$  in the BatchNorm1d class)

```

Code (no_grad because these are not optimised by backprop):

```
with torch.no_grad():
    bnmean_running = 0.999 * bnmean_running + 0.001 * bnmean_i
    bnstd_running = 0.999 * bnstd_running + 0.001 * bnstd_i
```

Numeric example — $\alpha = 0.001$ vs $\alpha = 0.1$:

α (momentum)	μ_{run} (before)	μ_{batch}	μ_{run} (after)	Comment
0.001	0.50	0.80	$0.50 \times 0.999 + 0.80 \times 0.001 = 0.5003$	Very stable
0.100	0.50	0.80	$0.50 \times 0.9 + 0.80 \times 0.1 = 0.530$	Faster adaptation

Training mode BN: uses batch mean/variance → couples samples together
Inference mode BN: uses `bnmean_running` / `bnstd_running` → consistent, batch-independent predictions

7 Tanh Nonlinearity & Gradient Saturation

$$h = \tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z}) \quad h \in (-1, +1)$$

$$d/dz \tanh(z) = 1 - \tanh^2(z)$$

Input z	$\tanh(z)$	Derivative $1 - \tanh^2(z)$	Gradient health
$z = 0.0$	0.000	1.000	Maximum — perfect
$z = 0.5$	0.462	0.786	Strong
$z = 1.0$	0.762	0.420	Moderate
$z = 2.0$	0.964	0.071	Weak — approaching saturation
$z = 3.0$	0.995	0.010	Near zero — vanishing
$z = 5.0$	0.9999	0.0002	Effectively dead

Saturation threshold in the code: `(t.abs() > 0.97)` means $|\tanh(z)| > 0.97$ which implies $|z| > 2.1$ and the derivative < 0.06 — less than 6% of the gradient survives. BN keeps z in $(-2, +2)$ by centring activations, making this rare.

8 Output Layer and Cross-Entropy Loss

$$\text{logits} = h \cdot W2 + b2 \quad W2 \in \mathbb{R}^{(200 \times 27)}, \quad b2 \in \mathbb{R}^{27}$$

$$L = -\left(\frac{1}{N}\right) \sum \log(\exp(\text{logits}_{\{b,yb\}}) / \sum \exp(\text{logits}_{\{b,j\}}))$$

Why W2 is initialised with scale 0.01: Small logits → near-uniform softmax → all 27 characters equally likely at start → initial loss $\approx -\log(1/27) \approx 3.3$. The notebook confirms: first printed loss is exactly 3.3239.

Why b2 is zero: No systematic bias at initialisation — cleaner baseline.

9 Full Forward Pass — One Formula

$$C[x] \rightarrow \text{concat} \rightarrow xW1 \rightarrow \text{BN}(\gamma, \beta, \mu, \sigma) \rightarrow \tanh \rightarrow hW2 + b2 \rightarrow \text{cross-entropy}$$

Step	Name	Formula	Output shape
①	Embed	$e = C[Xb]$	(32, 3, 10)
②	Concat	$X = \text{emb.view}(B, -1)$	(32, 30)
③	Linear	$h_{\text{pre}} = X \cdot W1$	(32, 200)
④	BN-norm	$h\text{-hat} = (h_{\text{pre}} - \mu) / \sqrt{\sigma^2 + \epsilon}$	(32, 200)
⑤	BN-affine	$h_{\text{BN}} = \gamma \cdot h\text{-hat} + \beta$	(32, 200)
⑥	Tanh	$h = \tanh(h_{\text{BN}})$	(32, 200)
⑦	Output	$\text{logits} = h \cdot W2 + b2$	(32, 27)
⑧	Loss	$L = \text{cross_entropy}(\text{logits}, y)$	scalar

10 Backpropagation Through the Full Stack

Step 1 — Loss → Logits:

$$\partial L / \partial \text{logits}_{\{b,j\}} = p_{\{b,j\}} - 1[j = y_b]$$

p is the softmax output. `F.cross_entropy` computes this automatically.

Step 2 — Through W2:

$$\partial L / \partial W2 = h \cdot \delta_{\text{logits}}$$

$$\partial L / \partial h = \delta_{\text{logits}} \cdot W2$$

Step 3 — Through Tanh:

$$\partial L / \partial h_{BN} = \partial L / \partial h \cdot (1 - \tanh^2(h_{BN}))$$

Saturated neurons multiply by ≈ 0 , killing their gradient. BN prevents widespread saturation, so most derivatives stay near 1.

Step 4 — Through BatchNorm (the hard part):

$$\begin{aligned}\partial L / \partial \gamma &= \sum_i \delta_{\{h_{BN}, i\}} \cdot \hat{h}_i \\ \partial L / \partial \beta &= \sum_i \delta_{\{h_{BN}, i\}} \\ \partial L / \partial h_{pre, i} &= (\gamma / \sqrt{(\sigma^2 + \epsilon)}) \cdot (\delta_i - (1/m) \sum \delta - (\hat{h}_i / m) \sum \delta \hat{h}_j)\end{aligned}$$

Key insight: Each sample's gradient depends on ALL other samples in the batch (through the $\sum \delta$ terms). This coupling is why BN helps stabilise training but also why very small batch sizes are problematic.

Step 5 — Through W1:

$$\begin{aligned}\partial L / \partial W1 &= X \cdot \delta_{\{h_{pre}\}} \\ \partial L / \partial X &= \delta_{\{h_{pre}\}} \cdot W1\end{aligned}$$

11 Parameter Update — SGD with LR Decay

$$p \leftarrow p - \eta \cdot \partial L / \partial p \quad \text{for every } p \text{ in parameters}$$

Learning rate schedule:

```
lr = 0.1 if i < 100000 else 0.01
```

5-step trajectory example ($W1[0,0]$ and γ):

Step	LR	$\partial L / \partial W1[0,0]$	$W1[0,0]$	$\partial L / \partial \gamma$	γ
0 (init)	—	—	0.500	—	1.000
1	0.1	+0.20	0.480	-0.043	1.004
2	0.1	+0.05	0.475	+0.020	1.002
3	0.1	-0.10	0.485	+0.015	1.001
4	0.01	+0.08	0.484	+0.030	0.998

5	0.01	+0.02	0.484	−0.010	0.999
---	------	-------	-------	--------	-------

12 Numerical Examples — BN Forward + Backward

Forward pass — 4 pre-activations through BN:

$$\mathbf{h_pre} = [5.0, 1.0, -3.0, 2.0]$$

Quantity	Formula	Value
μ	$(5+1-3+2)/4$	1.25
σ^2	$((5-1.25)^2+(1-1.25)^2+(-3-1.25)^2+(2-1.25)^2)/4$	8.19
σ	$\sqrt{8.19}$	2.86
$\hat{h}[0]$	$(5.0 - 1.25)/2.86$	+1.31
$\hat{h}[1]$	$(1.0 - 1.25)/2.86$	−0.09
$\hat{h}[2]$	$(-3.0 - 1.25)/2.86$	−1.49
$\hat{h}[3]$	$(2.0 - 1.25)/2.86$	+0.26
$h_BN[0]$	$1.5 \times 1.31 + 0.5 \quad (\gamma=1.5, \beta=0.5)$	+2.47
$h_BN[1]$	$1.5 \times (-0.09) + 0.5$	+0.37
$h_BN[2]$	$1.5 \times (-1.49) + 0.5$	−1.74
$h_BN[3]$	$1.5 \times 0.26 + 0.5$	+0.89
$h[0]$	$\tanh(2.47)$	0.986
$h[1]$	$\tanh(0.37)$	0.354
$h[2]$	$\tanh(-1.74)$	−0.942
$h[3]$	$\tanh(0.89)$	0.711

Backward pass — given $\delta_h_BN = [0.084, -0.042, -0.084, 0.168]$:

Gradient	Formula	Value
$\partial L / \partial \gamma$	$\sum \delta_i \cdot \hat{h_i}$	$0.084 \times 1.31 + (-0.042) \times (-0.09) + (-0.084) \times (-1.49) + 0.168 \times 0.26 = +0.314$
$\partial L / \partial \beta$	$\sum \delta_i$	$0.084 - 0.042 - 0.084 + 0.168 = +0.126$
$\partial L / \partial h_pre,0$	$(\gamma/\sigma)(\delta - (1/4)\sum \delta - h_hat \cdot (1/4)\sum \delta h_hat) - 0.015 \quad (\text{batch-coupled})$	

13 Custom Classes — PyTorchifying the Code

Class	Mimics	Parameters	Forward formula
Linear	nn.Linear	weight, (bias)	$y = x @ \text{weight} + \text{bias}$
BatchNorm1d	nn.BatchNorm1d	gamma, beta	$y = \text{gamma} \cdot (x - \mu) / \sigma + \text{beta}$
Tanh	nn.Tanh	none	$y = \tanh(x)$

Why bias=False in Linear when followed by BatchNorm? BatchNorm subtracts the batch mean, so any additive bias is immediately cancelled. Adding a bias would be redundant — BN's own β (bnbias) already does the shifting.

Comparison: manual inline BN vs BatchNorm1d class vs nn.BatchNorm1d:

Feature	Manual inline	BatchNorm1d class	nn.BatchNorm1d
Learnable params	bngain, bnbias	gamma, beta	weight, bias
Running stats	Manual EMA	Manual EMA	Automatic
Momentum α	0.001	0.1	0.1 (default)
Train/eval switch	Separate lines	self.training flag	model.train()/eval()
ϵ	1e-5 (via std)	1e-5	1e-5

14 Deeper 6-Layer Network

The notebook stacks 5 hidden blocks + one output block:

Layer block	Components	Input dim	Output dim	Parameters
Block 1	Linear + BN + Tanh	30	100	$30 \times 100 = 3000 + 200$
Block 2	Linear + BN + Tanh	100	100	$100 \times 100 = 10000 + 200$
Block 3	Linear + BN + Tanh	100	100	$10000 + 200$
Block 4	Linear + BN + Tanh	100	100	$10000 + 200$
Block 5	Linear + BN + Tanh	100	100	$10000 + 200$
Output	Linear + BN	100	27	$100 \times 27 = 2700 + 54$
Embedding	C	27	10	$27 \times 10 = 270$
Total				~47,024

Output layer special init: `layers[-1].gamma *= 0.1` shrinks the final BN's gamma so logits start near zero → near-uniform softmax → lower initial loss → more stable early training. Same principle as $W2 * 0.01$ in the simple version.

15 Inference & Name Sampling

Switching to eval mode:

```
for layer in layers: layer.training = False
```

This makes BatchNorm1d use running_mean and running_var instead of batch statistics.

Generation loop — one character at a time:

Step	Operation	Code	Math
1	Look up embeddings	<code>C[torch.tensor([context])]</code>	$e = C[ctx] \in \mathbb{R}^{(1 \times 3 \times 10)}$
2	Flatten context	<code>emb.view(1, -1)</code>	$x = \text{concat}(e) \in \mathbb{R}^{30}$
3	Forward through layers	<code>for layer in layers: x = layer(x)</code>	Apply each block
4	Softmax probabilities	<code>F.softmax(x, dim=1)</code>	$p_j = \exp(l_j) / \sum \exp(l_k)$
5	Sample next char	<code>torch.multinomial(probs, 1)</code>	$ix \sim \text{Cat}(p)$
6	Slide context	<code>context = context[1:] + [ix]</code>	window shifts right
7	Stop condition	<code>if ix == 0: break</code>	$ix=0 \rightarrow \text{'.' end-of-word}$

16 Linear vs Linear+BN — Activation Statistics

From notebook cells 23–24 (n=1000 neurons):

Quantity	Linear only	Linear + BN	Explanation
x std (forward)	31.0	31.0	$\sqrt{1000}$ blow-up from matmul
out std (forward)	31.0	1.0	BN normalises back to 1
x.grad std	0.97	0.032	BN compresses gradient too
inp.grad std	30.9	1.0	BN stabilises input grads
w.grad std	0.97	0.031	Weight gradients also scaled

Key takeaway: Without BN, a 1000-neuron linear layer amplifies std by $\sqrt{1000} \approx 31$. BN catches this in both directions — forward activations are normalised, and backward input gradients are also stabilised (~1 instead of ~31).

Matrix multiply gradient derivation (from cell 23 comments):

$$\mathbf{c} = \mathbf{b} \cdot \mathbf{a} \quad \blacksquare \quad \partial L / \partial \mathbf{a} = \mathbf{b} \blacksquare \cdot \partial L / \partial \mathbf{c} \quad | \quad \partial L / \partial \mathbf{b} = \partial L / \partial \mathbf{c} \cdot \mathbf{a} \blacksquare$$

Std of $\mathbf{a.grad} = \text{Std}(\mathbf{b} \blacksquare \cdot \mathbf{c.grad}) \approx n \times \text{Std}(\mathbf{b}) \times \text{Std}(\mathbf{c.grad})$ — which without scaling $\approx 1000 \times 1 \times 1 = 31$. BN prevents this blow-up by constraining $\text{Std}(\mathbf{c}) = 1$ before gradients flow backward.

Appendix — Parameter Count (Simple 1-hidden-layer version)

Tensor	Shape	Count
C (embeddings)	27×10	270
W1	30×200	6,000
W2	200×27	5,400
b2	27	27
bngain (γ)	1×200	200
bnbias (β)	1×200	200
Total		12,097

The deeper 6-layer version has **47,024** parameters.

Quick Reference — Loss Improvement History

Version	Train loss	Val loss	Change made
Original MLP	2.125	2.168	Baseline
Fix softmax init	2.07	2.13	$W2 * 0.01$
Fix tanh saturation	2.036	2.103	Better W1 init
Kaiming init	2.038	2.107	$(5/3)/\sqrt{n}$ factor
+ Batch Norm	2.067	2.105	BN after W1